WAR ROOM COUNCIL

Mitigata Hackathon 2026 — The Hackathon Bible

Mitigata Hackathon 2026 · 31 October 2026

Contents

Mitigata Hackathon 2026 — The Hackathon Bible
Part 1 — The Facts
  At a glance
  Evaluation criteria
  Themes & problem statements
  Timeline & deliverables checklist
Part 2 — The People
  Organisers & committee
  Sponsors
  Judges & jury
  Judge heat-map
Part 3 — Why This Hackathon, Why Now
  The real reason it exists
  Policy & scheme drivers
  Incidents & history behind the problem statements
  Geopolitical lens (global + India)
  What the organisers want to announce afterwards
Part 4 — Connecting the Dots
  Dots nobody else will connect
  Hidden evaluation criteria
  Landmines — what will annoy this jury
  The 80% trap — what most teams will build
Part 5 — The Battlefield
  Competition forecast
  Prior work by domain
  Domain Winnability ranking
Part 6 — The Council
  Who sat in the war room
Sources

Mitigata Hackathon 2026 — The Hackathon Bible

> **One-Paragraph Verdict: Mitigata Hackathon 2026** is not a generic college coding contest—it is a high-signal R&D and talent-acquisition pipeline for **Mitigata** (`https://mitigata.com/` , India's First Insurtech Platform for Cyber Resilience, IRDAI Direct Broker Reg. `1115/2024` , backed by a $15M Series B led by Bessemer Venture Partners alongside Nexus Venture Partners and Titan Capital) [S1, S4, S11]. Every one of the six official problem statements (`PROBLEM 01` through `PROBLEM 06 · FLAGSHIP`) maps 1-to-1 to an active engineering epic inside **`Gordon AI (GORDON-V3)`** (`Monitor · Assess · Mitigate · Insure`) [S1, S6]. Above all, **`PROBLEM 06 · FLAGSHIP`** **— SAST and DAST Correlation** is the **only** challenge badged **`FLAGSHIP · Solve it for bonus points`** on Slide `07 / 07` and backed by a dedicated 5-page quantitative PRD authored by **`Anuj`** (`30 October 2026`) [S1, S2]. To win outright, avoid the 80% trap of gluing Semgrep and OWASP ZAP together with an LLM prompt (which fails the `Wrong links: Under 5%` and `False alarms: Under 10%` targets) and instead build **`GORDON CORRELATE (CorrelAid Engine)`** : a deterministic **AST Route-to-Sink Call-Graph (`FR-4`) + 2-Phase Authenticated DAST (`FR-2`) + `B-1` Active Auto-Verifier + `B-2` Safe-Decoy Killer + 5-Step Reachability-Aware SCA (`Page 4 Bonus`)** that links all 20 planted pairs with **0% wrong links**, suppresses all **8 safe decoys**, captures all **6 runtime-only bugs**, generalises to the **unseen second app (`B-7`)**, and finishes in **under 35 seconds** [S2].

Part 1 — The Facts

At a glance

Field	Verified Fact & Source
Event Name	**Mitigata Hackathon 2026** ("*Six challenges across security, data and product engineering*") [S1]
Organiser & Host	**Mitigata** (mitigata.com) — **Mitigata Insurance Brokers Pvt. Ltd.** (wholly owned subsidiary of **Alphawave Technologies Private Limited**), Bellandur, Bengaluru [S4, S11]
Flagship PRD Author & Date	**Anuj** · `30 October 2026` (`PRD: SAST and DAST Correlation — Problem, Goals and Requirements`) [S2]
Official Source Artefacts	1. `Hackathon Problem Statements.pdf` (7 slides, `01 / 07` to `07 / 07`, covering `PROBLEM 01` to `PROBLEM 06 · FLAGSHIP + INC-01` to `INC-06`) [S1] 2. `PRD_SAST_DAST_Correlation.docx.pdf` (5-page Flagship specification & answer-key grading rubric for `PROBLEM 06`) [S2] 3. `Problem Statements.docx` (6-challenge initial summary draft, dated `29 October 2026`) [S3]
Selected Track	`PROBLEM 06 · FLAGSHIP` **— SAST and DAST Correlation (+ Reachability-Aware SCA)** ("*Flagship Challenge. Any team that solves this problem earns bonus points on top of its normal score.*") [S1, S2]
Kickoff Test Application	**One provided test app:** A small web shop with **~30 URLs**, full source code, and a **Docker Compose** file to run locally, containing **36 planted vulnerabilities** (`20 pairs + 10 Code only + 6 Runtime only`) and **~8 safe decoys** [S2]
Judging Evaluation App	**Unseen second app:** At judging, the tool also runs on a second web application with the same kinds of bugs to verify it is not tuned only to the kickoff test app (`Great target: Over 65%` pairs linked) [S2]
Submission Format (`FR-10`)	Simple command line, REST API (`POST /scans` , `GET /issues` , `GET /issues/7`), or web page, with a clear `README` (Docker Compose is a plus) listing all open-source scanners/rules used [S2]
Hard Disqualification Rule	"*Scan only the test app. Point the DAST scanner only at the organisers' app running on your own machine. Scanning any other site or system is not allowed and leads to disqualification.*" [S2]

Evaluation criteria

Stated Quantitative Targets for `PROBLEM 06 · FLAGSHIP` (Measured Against the Answer Key — PRD Page 4)

The organisers grade `PROBLEM 06 · FLAGSHIP` against **7 objective answer-key metrics** (`hit the Good column to qualify and the Great column to stand out`) plus **Correlation quality** ("*Correlation is what we judge most*" — Slide 7 & PRD Page 1), **Bonus items (`B-1` to `B-7`)**, and the **Page 4 Bonus Feature (`Reachability-Aware SCA`)** [S1, S2]:

What we measure	How (Verbatim from PRD Page 4)	Good (Qualify)	Great (Stand Out)	Our Architecture Target (GORDON CORRELATE)
Code detection	Share of planted code vulnerabilities your SAST finds (out of 30 code bugs: 20 pairs + 10 code-only)	Over 60% (`>= 19/30`)	**Over 85%** (`>= 26/30`)	**93%+ (`>= 28/30`)** via AST taint + Semgrep/Bandit + `B-2` decoy guard
Runtime detection	Share of planted runtime vulnerabilities your DAST finds (out of 26 runtime bugs: 20 pairs + 6 runtime-only)	Over 50% (`>= 14/26`)	**Over 80%** (`>= 21/26`)	**92%+ (`>= 24/26`)** via 2-Phase DAST (AST-seeded active + `B-1` + passive header audit)
Correct links	Share of the 20 planted pairs linked into one `Confirmed` issue	Over 50% (`>= 11/20`)	**Over 80%** (`>= 17/20`)	**90%–100% (`18–20 / 20`)** via `FR-4` AST Route Map + Call Graph + `B-1` auto-verify
Wrong links	Share of your `Confirmed` issues that join two different bugs	Under 20%	**Under 5%**	**0.0% (`0 wrong links`)** via hard `(CanonicalRoute, Method, Param/Sink, CWE)` vetoes
False alarms	Share of reported issues that are safe decoys (out of ~8 safe decoys)	Under 30%	**Under 10%**	**0.0% (`0/8 decoys reported`)** via `B-2` AST parameterized-query & guard verifier
Unseen app	Share of planted pairs linked on the second app at judging	Over 40%	**Over 65%**	**75%–85%+** via zero-hardcoding polyglot Route-to-Sink IR (`Python + Express B-7`)
Run time	Time to scan, correlate and produce the report on the test app	Under 15 minutes	**Under 5 minutes**	**25–40 seconds** via in-process `ast` / `tree-sitter` (`<1s`) + async `httpx` (`~25s`)

Complete Verbatim Functional Requirements (`FR-1` to `FR-10` — PRD Pages 2–3)

ID	Feature (Verbatim from PRD)	Example (Verbatim from PRD)
`FR-1`	SAST scan: scan a source folder and report findings with file, line, vulnerability type, severity and the code snippet	`SQL injection, orders.py line 88, CWE-89`

ID	Feature (Verbatim from PRD)	Example (Verbatim from PRD)
`FR-2`	DAST scan: scan a running app URL by sending test requests and report findings with URL, method, parameter, payload and evidence	`GET /api/orders?id=1' returns a database error`
`FR-3`	One common finding format for both scanners, with a CWE number for the type	`Both sides report SQL injection as CWE-89`
`FR-4`	Route map: read the source code to list which URL is handled by which function and lines	`GET /api/orders -> get_orders(), orders.py lines 80 to 95`
`FR-5`	Correlate: link a DAST finding to a SAST finding when they have the same CWE and the DAST URL is handled by the code that holds the SAST finding	`DAST SQL injection on /api/orders + SAST SQL injection at orders.py:88 = one confirmed issue`
`FR-6`	Merge duplicates: one issue per real bug with all its evidence attached; running a scan twice does not create copies	`3 scanner findings on the same bug = 1 issue`
`FR-7`	Label every issue as `Confirmed` (found by both), `Code only` (SAST, not proven) or `Runtime only` (DAST, no code location found)	`Hard-coded secret = Code only`
`FR-8`	Issue detail for confirmed issues: URL, parameter, payload, file, line, code snippet, CWE and severity	`One page that tells a developer where to look`
`FR-9`	Prioritised list: confirmed issues first, then by risk score (severity and how easy it is to exploit)	`Confirmed SQL injection above a Code only weak hash`
`FR-10`	Simple command line, API or page, with a short README	`POST /scans, GET /issues, GET /issues/7`

Complete Verbatim Bonus Items (`B-1` to `B-7` — PRD Page 3) & Six CWE Vulnerability Types

Bonus ID	Feature (Verbatim from PRD Page 3)	CWE Coverage (PRD Page 3: Cover >= 4, we cover all 6)
`B-1`	**Auto-verify:** for a `Code only` finding, send a safe test request to try to prove it at runtime and upgrade it to `Confirmed`	`CWE-89` **— SQL injection** (`Code and runtime`)
`B-2`	**Cut SAST noise:** mark findings as safe when the code is protected (e.g. a parameterised query) or no URL can reach it	`CWE-79` **— Cross-site scripting (XSS)** (`Code and runtime`)
`B-3`	**Explain every link:** show why two findings were joined (same CWE, same route, same parameter)	`CWE-78` **— Command injection** (`Code and runtime`)
`B-4`	**Plain-English fix suggestion** for each confirmed issue	`CWE-22` **— Path traversal** (`Code and runtime`)
`B-5`	**Compare two scans** and show new, fixed and still-open issues	`CWE-601` **— Open redirect** (`Code and runtime`)

Bonus ID	Feature (Verbatim from PRD Page 3)	CWE Coverage (PRD Page 3: Cover >= 4, we cover all 6)
`B-6`	**Export results as SARIF** and fail a CI build on confirmed high-severity issues	`CWE-798` **— Hard-coded secrets** (`Code only`)
`B-7`	**Support a second programming language or framework**	Plus runtime header/cookie/CORS checks for the **6** `Runtime only` **bugs**

Page 4 Special Bonus Feature: Reachability-Aware SCA & Page 5 Strategic Question

- **Page 4 Verbatim Bonus Specification (** `NOTE : Bonus feature: reachability-aware SCA` **):**

  > *"Report whether the app can actually reach a vulnerable library function, not just that a CVE exists. 1.* ***Inventory:*** *Parse lockfiles into an SBOM, keeping transitive paths. 2.* ***GHSA match:*** *Pull affected versions, fixes, CVSS, EPSS and KEV. 3.* ***Function map:*** *Build it from OSV symbols and fix-commit diffs, since GHSA rarely lists functions. 4.* ***Call graph:*** *Link entry points, app functions and library functions (CodeQL, Joern, Semgrep). 5.* ***Supply chain:*** *Check for malicious packages, typosquats, dependency confusion and unpinned lockfiles.* ***Statuses:*** `Reachable` , `Imported-only` , `Present-only` , `Unknown` *(never downgrade* `Unknown` *)."* [S2]

- **Page 5 Verbatim Prompt:** "*Which language should this target first?*" — Answered definitively by PRD evidence (`orders.py` , `get_orders()` , `Python's ast module or tree-sitter`): target **Python (Flask / FastAPI / Django) first**, and **JavaScript / TypeScript (Node.js Express) second** via `tree-sitter` for Bonus `B-7` [S2].
- **Our Read of the Hidden Weights:**
- **45% — Answer-Key Correlation Accuracy (** `Correct links > 80%` **,** `Wrong links < 5%` **,** `False alarms < 10%` **):** Automated scoring against the 36-bug + 8-decoy answer key and the unseen second app is the primary filter.
- **25% — Route Map (** `FR-4` **),** `B-1` **Auto-Verify &** `B-2` **Safe Decoy Suppression:** These three mechanisms determine whether a team hits the `Great` column across both static and dynamic metrics.
- **15% — Page 4 Reachability-Aware SCA Bonus (5 Steps +** `Never downgrade Unknown` **) &** `B-3` **–** `B-7` **:** Separates the #1 Flagship winner from teams that only implement basic SAST+DAST.
- **15% — Speed (** `< 5 min` **), Idempotency (** `FR-6` **), Developer UX (** `FR-8` **–** `FR-10` **) & Mitigata Gordon AI Fit:** Instant Docker Compose start, sub-minute scan time, and alignment with Mitigata's `/security/dast-sast` and `/gordon-ai/vapt` workflow.

Themes & problem statements

All 6 official challenges from `Hackathon Problem Statements.pdf` (Slides `01 / 07` to `07 / 07`) [S1]:

ID & Slide	Title	Scope / Constraint Pill	Three Official Deliverables (01, 02, 03)	Real Incident Anchor (INC-01 to INC-06)
`PROBLEM 01` *(Slide 02/07)*	**Kill chain and attack path analysis**	`Data tiers: 15, 30 or 50 GB of logs`	**01 Detect and label:** Find suspicious events and tag each with a MITRE ATT&CK technique and kill chain stage. **02 Connect the steps:** Link events across hosts, users, IPs and time into one attack path, without running out of memory. **03 Explain and look ahead:** Show a timeline and a graph of hosts and users, plus what the attacker could reach next.	`INC-01` **:** "*A 2013 retail breach: one stolen vendor login, weeks of ignored alerts, up to 110 million customers.*" (2013 Target HVAC vendor breach) [S1]
`PROBLEM 02` *(Slide 03/07)*	**Multi-tenant aggregation layer**	`Scale: 10M records, 100 tenants`	**01 Fast and fresh:** Dashboard queries return in under a second, and new or updated orders show up within minutes. **02 Four dashboard queries:** Counts and totals, trends over time, top-N lists and breakdowns by category, through a simple API. **03 Separate and correct:** Each tenant sees only its own data, and the numbers match a direct database query.	`INC-02` **:** "*A 2023 cache bug that showed some users other users' chat titles and payment details.*" (March 2023 OpenAI ChatGPT Redis bug) [S1]
`PROBLEM 03` *(Slide 04/07)*	**Community scam map**	`Reporters stay anonymous`	**01 Report and look up:** Report a scam in under 60 seconds. Lookups answer in under half a second. **02 Map and alerts:** Show the last 7 days of reports by area, and alert only when something is unusual. **03 Beat fake reports:** Trust rules must stop report-bombing, fake accounts, spam and junk, without missing real scams.	`INC-03` **:** "*India's 'digital arrest' scams: over 83,000 WhatsApp accounts blocked by March 2025.*" (I4C / MHA crackdown) [S1]

ID & Slide	Title	Scope / Constraint Pill	Three Official Deliverables (01, 02, 03)	Real Incident Anchor (INC-01 to INC-06)
`PROBLEM 04` *(Slide 05/07)*	**Autonomous pentest engine (CART)**	`Test only inside the given scope`	**01 Discover all three surfaces:** Find live hosts, ports and services, then crawl web apps and inspect the binaries. **02 Coordinate agents:** Agents share findings and hand work to each other, so no host is tested twice. **03 Verify and report:** Prove each weakness, tag it with a PTES phase, and deliver one report with the attack path.	`INC-04` **:** "*A 2019 bank breach: a misconfigured web firewall led to cloud data on about 100 million people.*" (2019 Capital One AWS WAF SSRF breach) [S1]
`PROBLEM 05` *(Slide 06/07)*	**Config-driven integration platform**	`Prove it with 3 different sources`	**01 Config describes the source:** Login, endpoints, pagination, schedule, and a mapping into one common format. **02 Platform handles the rest:** Safe secrets, retries with backoff, rate limits and bad data, the same for every connector. **03 Health stays visible:** A simple page shows each connection's status, last run and errors.	`INC-05` **:** "*A 2025 breach where stolen integration tokens exposed many companies' CRM data.*" (2025 Salesloft Drift OAuth token breach) [S1]
`PROBLEM 06 · FLAGSHIP` *(Slide 07/07)*	**SAST and DAST correlation**	`Solve it for bonus points`	**01 Scan code and app:** Cover at least four of six vulnerability types, including SQL injection and XSS, in one common format with CWE numbers. **02 Correlate and merge:** Link a runtime finding to the code behind it and merge duplicates. This is what we judge most. **03 Prioritise what to fix:** Label each issue Confirmed, Code only or Runtime only, and rank confirmed issues first.	`INC-06` **:** "*A 2017 credit bureau breach: a known web flaw that scans missed, about 145 million people.*" (2017 Equifax Apache Struts CVE-2017-5638 breach) [S1, S12]

Timeline & deliverables checklist

- **Phase 0 — Kickoff & PRD Lock (`30–31 October 2026`):**
  - [x] Analyse all 7 slides of `Hackathon Problem Statements.pdf` (`PROBLEM 01` to `PROBLEM 06 · FLAGSHIP`) and all 5 pages of `PRD_SAST_DAST_Correlation.docx.pdf` (authored by `Anuj`) [S1, S2].
  - [x] Crawl `https://mitigata.com/` (`/security/dast-sast`, `/gordon-ai`, `/gordon-ai/vapt`, `/gordon-ai/cyber-risk-quantification`, `/compliance/sebi-cscrf`) and align council & architecture [S4–S11].
- **Phase 1 — Hours 0–6 (Common Schema, Polyglot AST Route Map & Idempotent Store):**
  - [ ] Implement canonical CWE finding & issue schema (`FR-3`, `FR-7`) and SQLite/PostgreSQL idempotent fingerprint store (`FR-6`: running a scan twice never creates copies).
  - [ ] Build `ast` + `tree-sitter` Route Mapper (`FR-4`) + intra/inter-module Call Graph supporting Python (`Flask`, `FastAPI`, `Django`) first and Node.js (`Express`) second (`B-7`).
- **Phase 2 — Hours 6–14 (Precision SAST + `B-2` Decoy Killer & 2-Phase DAST + `B-1` Auto-Verifier):**
  - [ ] Implement 6-CWE SAST scanner + Semgrep/Bandit adapter (`FR-1`: `CWE-89`, `CWE-79`, `CWE-78`, `CWE-22`, `CWE-601`, `CWE-798`) with `B-2` AST Protection & Reachability Verifier to suppress all ~8 safe decoys (`0% False Alarms`).
  - [ ] Implement Scope-Locked 2-Phase Async DAST (`FR-2`: localhost/container target lock, `follow_redirects=False`) combining AST-seeded active CWE probes + `B-1` Auto-Verify (`Code only` → `Confirmed`) + Phase B passive runtime header/cookie/CORS auditor (`6 Runtime only` bugs).
- **Phase 3 — Hours 14–22 (Deterministic Correlator & 5-Step Reachability-Aware SCA Bonus):**
  - [ ] Implement `FR-5` Multi-Signal Correlator with hard negative vetoes (`Wrong links = 0% < 5%`), `B-3` link explanation badges, `FR-9` risk score ranking (`Confirmed` first + FAIR `₹` exposure), and `B-4` plain-English fix suggestions.
  - [ ] Implement all 5 steps of Page 4's **Reachability-Aware SCA** (Transitive Lockfile SBOM → GHSA/CVSS/EPSS/KEV → OSV Symbols + GitHub Fix-Commit Diff Function Map → Entry-to-Library Call Graph → Supply-Chain Typosquat/Confusion/Unpinned checks, preserving `Unknown`).
- **Phase 4 — Hours 22–30 (API/CLI/UI, `B-5` Scan Diff, `B-6` SARIF Gate & Unseen App Validation):**
  - [ ] Expose `POST /scans`, `GET /issues`, `GET /issues/7` (`FR-10`), single-page issue inspector (`FR-8`), `B-5` Scan-to-Scan Comparator (`new`, `fixed`, `still-open`), `B-6` SARIF v2.1.0 export + CI build breaker, and Docker Compose `README` disclosing all open-source tools used.

Part 2 — The People

Organisers & committee

Entity / Person	Role & Credentials	Mandate & Motive in This Hackathon	Evidence
Mitigata Insurance Brokers Pvt. Ltd. *(Subsidiary of Alphawave Technologies Pvt. Ltd.)*	Organiser · Headquartered in Bellandur, Bengaluru (offices in Mumbai, Gurugram, Delaware USA); IRDAI Direct Broker Reg. `1115/2024`, License `1013`; CIN `U66220KA2024PTC186580`	Building India's unified **Cyber Resilience + Smart Cyber Insurance** platform (`Gordon AI · GORDON-V3`). Uses the hackathon to solve six production roadmap challenges and recruit top security/systems engineers.	[S1, S4, S11]
Mohit Anand	Co-Founder & CEO, Mitigata	Connects technical security posture (`Assess`) directly to financial risk quantification (`₹ Cr`) and insurance underwriting (`Insure`) across 800+ clients and 18+ IRDAI insurers.	[S4, S8, S11]
Sarthak Dubey	Co-Founder & COO, Mitigata	Leads operational delivery across Mitigata's VAPT, MDR, and GRC pods; owns the `/gordon-ai/vapt` promise of **`0 False positives`**, **`38 Mean findings`**, and **`48h Report delivery`**.	[S7, S11]
Mayank Morya	Co-Founder & CTO, Mitigata	Chief Architect of **`Gordon AI (GORDON-V3)`**; owns the engineering architecture across all 6 hackathon challenges (AI SOC, multi-tenant rollups, CART pentest, integrations, SAST/DAST correlation).	[S6, S11]
Akshit Kaushik	Co-Founder & CBO, Mitigata	Leads enterprise growth across BFSI, Fintech, SaaS, and Healthcare enterprises requiring SEBI CSCRF, RBI, and DPDP compliance.	[S9, S11]

Entity / Person	Role & Credentials	Mandate & Motive in This Hackathon	Evidence
Lt Cdr Dheeraj Yadav & Dr. Lalit Gupta	CISO (ex-Deputy Director, Indian Navy) & Senior Partner / CDO (Founding Member, CERT-In India)	Enforce national-grade security rigour, CERT-In compliance, and zero-false-positive audit evidence standards.	[S11]
Anuj	Author of `PRD_SAST_DAST_Correlation.docx.pdf` (`30 October 2026`)	Designed `PROBLEM 06 · FLAGSHIP` , the 36-bug + 8-decoy web shop, the unseen judging app, the 7 quantitative targets, and the 5-step Reachability-Aware SCA bonus spec.	[S2]

Sponsors

- **Primary Corporate Sponsor: Mitigata (`https://mitigata.com/`)** — Backed by **$15 Million in Series B funding** led by **Bessemer Venture Partners (BVP)**, alongside **Nexus Venture Partners**, **Titan Capital**, and **WEH Ventures**, and named to **Forbes Asia 100 to Watch 2026** [S4, S11].
- **What Mitigata Sells:** 1. **Gordon AI (`GORDON-V3`):** Unified platform across `Monitor · Assess · Mitigate · Insure` [S6]. 2. **Application Security Testing (`/security/dast-sast`) & Autonomous VAPT (`/gordon-ai/vapt`):** Continuous SAST, DAST, API security, and PTES pentesting with a 4-stage pipeline (`MAP -> SELECT -> TUNE -> VERIFY`) [S5, S7]. 3. **Cyber Risk Quantification (`/gordon-ai/cyber-risk-quantification`) & Compliance (`/compliance/sebi-cscrf`):** FAIR-based financial risk scoring in `₹` and automated compliance across **SEBI CSCRF, RBI, IRDAI, DPDP Act 2023, ISO 27001, SOC 2, PCI-DSS** [S8, S9]. 4. **Smart Cyber Insurance Broking:** Active risk-backed policies with 18+ Indian insurers and `₹500 Cr` in settled claims [S4].
- **How to Win Their Flagship Prize:** Build `PROBLEM 06 · FLAGSHIP` so that it looks and behaves like a native module of `GORDON-V3` —delivering `0%` false alarms on the 8 safe decoys, `0%` wrong links on the 20 pairs, `B-1` auto-verification (`Find it. Prove it. Fix it. Re-scan it.`), and `FR-9` risk prioritisation enriched with FAIR `₹` exposure and SEBI CSCRF SLA badges.

Judges & jury

1. **Anuj (Flagship PRD Author & Lead Technical Evaluator):** - *Background:* Senior AppSec / Systems Architect at Mitigata who authored the 5-page `PRD_SAST_DAST_Correlation.docx.pdf` on October 30, 2026 [S2]. - *What they reward:* Exact compliance with `FR-1` to `FR-10` , hitting the **Great** column on all 7 answer-key targets (especially `Correct links > 80%` , `Wrong links < 5%` , `False alarms < 10%` , `Unseen app > 65%` , `Run time < 5 min`), plus `B-1` – `B-7` and the 5-step **Reachability-Aware SCA** bonus (`OSV symbols + fix-commit diffs` , `Never downgrade Unknown`) [S2]. - *Likely question:* "*Show me how your `FR-4` route map links `GET /api/orders` to `orders.py:88` when the SQL query is inside a helper function, how you avoid flagging the 8 parameterized-query decoys (`B-2`), and how your tool performs on our unseen second app.*"

2. **Mayank Morya (Co-Founder & CTO, Mitigata — Architect of `GORDON-V3`):** - *Background:* Systems & AI architect scaling Gordon AI across 800+ clients and 1M+ assets [S6, S11]. - *What they reward:* Clean, deterministic graph engineering (`ast` + `tree-sitter` + async `httpx`), idempotent database design (`FR-6`), sub-minute speed, and zero-friction Docker Compose deployment. - *Likely question: "What happens when I run `POST /scans` twice on the same app (`FR-6`), or run your scanner offline without external LLM API keys?"*
3. **Sarthak Dubey (Co-Founder & COO, Mitigata — VAPT & Operations Lens):** - *Background:* Runs Mitigata's VAPT and SOC delivery pods (`0 False positives`, `38 Mean findings`, `48h delivery`) [S7, S11]. - *What they reward:* `B-1` Auto-Verify (turning `Code only` guesses into proven `Confirmed` exploits), `B-4` plain-English fix suggestions, `B-5` scan comparison (`new`, `fixed`, `still-open`), and `B-6` SARIF CI/CD gating. - *Likely question: "After a developer applies your `B-4` fix suggestion and re-scans, how does `B-5` prove to a client that the issue is fixed?"*
4. **Mohit Anand (Co-Founder & CEO) & Lt Cdr Dheeraj Yadav / Dr. Lalit Gupta (CISO / CERT-In Delivery Lens):** - *Background:* Executive leadership bridging technical VAPT findings to boardroom risk (`₹ Cr`), SEBI CSCRF / CERT-In compliance, and IRDAI cyber insurance underwriting [S4, S8, S9, S11]. - *What they reward:* A crystal-clear `FR-9` prioritised list where `Confirmed` exploitable issues sit at the top with clear business/compliance impact. - *Likely question: "Why should a CISO or cyber insurance underwriter trust your `FR-9` risk score over a raw Semgrep or ZAP dump?"*

Judge heat-map

Judge / Evaluator Persona	Primary Obsession	What Will Lose Their Vote	Our Winning Angle (GORDON CORRELATE)
Anuj *(Flagship PRD Author)*	**Answer-key precision:** `Wrong links < 5%` (`0` errors), `False alarms < 10%` (`0` decoys), `Unseen app > 65%`, and 5-step Reachability SCA	LLM prompt wrappers that hallucinate 1–2 wrong links (`1/20 = 5%` fails Great!) or regex rules that report the 8 safe decoys	Deterministic `FR-4` AST Route-to-Sink Call Graph + hard negative vetoes (`0%` wrong links) + `B-2` AST guard verifier (`0%` decoys) + OSV/fix-commit diff SCA
Mayank Morya *(CTO & Gordon AI Architect)*	**Speed, idempotency & graph soundness:** `< 5 min` runtime, `FR-6` duplicate merge, clean API (`POST /scans`, `GET /issues/7`)	Slow 12-minute headless ZAP/CodeQL wrappers, non-idempotent DB inserts that duplicate issues on re-scan	`< 35s` total scan time (`ast` / `tree-sitter` in `<1s` + async `httpx` in `~25s`) + SHA-256 idempotent upsert store (`FR-6`)
Sarthak Dubey *(COO & VAPT Pod Lead)*	**Workflow closure (`/security/dast-sast`):** `MAP -> SELECT -> TUNE -> VERIFY`, `B-1` auto-verify, `B-4` fix, `B-5` diff, `B-6` SARIF	Dumping separate SAST and DAST tables without `B-3` link explanations or `B-5` re-scan verification	Live `B-1` Auto-Verify upgrading `Code only` → `Confirmed`, `B-3` visual bridge (`CWE + Route + Param`), `B-4` plain-English fix, and `B-5` Scan Diff

Judge / Evaluator Persona	Primary Obsession	What Will Lose Their Vote	Our Winning Angle (GORDON CORRELATE)
Mohit Anand & Dr. Lalit Gupta *(CEO & CERT-In CDO)*	**Actionable priority (`FR-9`):** Connecting `Confirmed` exploits (`INC-06` prevention) to `₹` risk, SEBI CSCRF SLAs & insurance underwriting	Toy risk scores (`severity * 2`) with no business, compliance, or underwriting context	Dual-layer `FR-9` Risk Score: Technical Exploitability (`Confirmed > Runtime only > Code only`) + Gordon AI FAIR `₹` Exposure & SEBI CSCRF SLA badge

Part 3 — Why This Hackathon, Why Now

The real reason it exists

Mitigata raised its **$15M Series B led by Bessemer Venture Partners** on a bold thesis: Indian enterprises do not want 10 disconnected security point-tools and a separate insurance broker—they want **one autonomous AI platform (`Gordon AI · GORDON-V3`)** that monitors, assesses, mitigates, and underwrites cyber risk [S4, S6, S11]. However, inside the `Assess` pillar (`/security/dast-sast` and `/gordon-ai/vapt`), correlating static source code findings (`SAST`) with live HTTP exploit proofs (`DAST`) and dependency reachability (`SCA`) is the single highest-leverage automation problem [S2, S5, S7]. By making `PROBLEM 06` **the `FLAGSHIP` challenge** with a 36-bug web shop benchmark modeled directly on Mitigata's internal VAPT baseline (`38 Mean findings`, `0 False positives` on `/gordon-ai/vapt`), Mitigata is testing for an engine and engineering team that can plug directly into `GORDON-V3` [S2, S7].

Policy & scheme drivers

1. **SEBI CSCRF (`SEBI/HO/ITD-1/ITD_CSC_EXT/P/CIR/2024/113`, Mandatory Enforcement 2025–2026):** Mandates all SEBI Regulated Entities (brokers, mutual funds, depositories, fintechs) to perform comprehensive SAST, DAST, and API VAPT via CERT-In empanelled auditors, **close all identified vulnerabilities within 3 months**, and complete **revalidation audits within 5 months** (`B-5` scan comparison) [S9, S13].
2. **Digital Personal Data Protection (DPDP) Act 2023 & DPDP Rules 2025:** Section 8(5) mandates "reasonable security safeguards" by every Data Fiduciary; failure resulting in a personal data breach carries statutory penalties up to **₹250 Crore** [S15]. Proving that public-facing web routes (`FR-4`) touching PII tables are free of `Confirmed` SQLi/XSS/Path Traversal is now a board-level legal defense.
3. **CERT-In Cyber Security Directions (April 2022):** Mandates **6-hour incident reporting**, 180-day log retention in India (`PROBLEM 01` & `PROBLEM 02`), and SBOM / third-party software supply-chain hygiene (`PROBLEM 06` Page 4 SCA Bonus) [S14].
4. **RBI Cyber Security Framework & IRDAI Information and Cyber Security Guidelines:** Require continuous vulnerability management, patch verification, and objective risk scoring for cyber insurance underwriting [S4, S8].

Incidents & history behind the problem statements

Every problem statement in `Hackathon Problem Statements.pdf` is explicitly anchored to a real-world historical catastrophe (`REAL INCIDENT · INC-01` to `INC-06`) [S1]:

PS ID	Incident Badge (Hackathon Problem Statements.pdf)	Historical Breach & Root Cause Evidence	The Organiser's Real Worry	What Our Winning Answer Must Prove
`PROBLEM 01`	`INC-01` : *"A 2013 retail breach: one stolen vendor login, weeks of ignored alerts, up to 110 million customers."*	**2013 Target Breach:** Attackers pivoted from an HVAC vendor's credentials into POS registers; FireEye/SIEM alerts fired for weeks but were ignored as isolated low-priority events [S1].	SIEMs drown SOC analysts in isolated alerts instead of connecting multi-host kill chains across 50 GB of logs.	Streaming MITRE ATT&CK graph correlation that links low-signal events into one narrative without OOM crashes.
`PROBLEM 02`	`INC-02` : *"A 2023 cache bug that showed some users other users' chat titles and payment details."*	**March 2023 OpenAI ChatGPT `redis-py` Bug:** Async cancellation in a shared Redis connection pool returned Tenant A's cached responses to Tenant B [S1].	Multi-tenant SaaS dashboards either overload the primary DB or leak cross-tenant data via shared caches.	Sub-second `<1s` pre-aggregated rollups across 10M rows & 100 tenants with cryptographic tenant-key isolation.
`PROBLEM 03`	`INC-03` : *"India's 'digital arrest' scams: over 83,000 WhatsApp accounts blocked by March 2025."*	**2024–2025 Indian Digital Arrest Wave (I4C / MHA):** Scammers impersonating police/CBI/ED extorted thousands of crores via UPI and spoofed numbers before blocklists updated [S1].	Victims need `<0.5s` pre-transaction lookup of phone/UPI/links, while attackers try to poison crowdsourced maps with fake reports.	Anonymous `<60s` reporting, `<500ms` lookup, 7-day spatio-temporal spike alerts, and Sybil/report-bombing resistance.
`PROBLEM 04`	`INC-04` : *"A 2019 bank breach: a misconfigured web firewall led to cloud data on about 100 million people."*	**2019 Capital One Breach:** An AWS WAF SSRF misconfiguration allowed an attacker to query the EC2 metadata service (`169.254.169.254`), assume an IAM role, and dump S3 buckets [S1].	Single-surface scanners miss multi-hop attack paths spanning network ports, web apps, and binary services.	Coordinated multi-agent PTES engine (web + network + binary) with a shared state blackboard so no host is tested twice.
`PROBLEM 05`	`INC-05` : *"A 2025 breach where stolen integration tokens exposed many companies' CRM data."*	**2025 Salesloft Drift OAuth Token Breach:** Compromised third-party integration tokens allowed attackers to exfiltrate Salesforce/CRM data across hundreds of enterprise tenants [S1].	Custom-coded connectors break on rate limits/schema drift and store OAuth/API secrets unsafely.	Declarative YAML/JSON connector engine with KMS-isolated secrets, exponential backoff, DLQ, and live health UI.

PS ID	Incident Badge (Hackathon Problem Statements.pdf)	Historical Breach & Root Cause Evidence	The Organiser's Real Worry	What Our Winning Answer Must Prove
`PROBLEM 06 · FLAGSHIP`	`INC-06` : *"A 2017 credit bureau breach: a known web flaw that scans missed, about 145 million people."*	**2017 Equifax Breach (`CVE-2017-5638`):** Apache Struts RCE went unpatched for 76 days because static dependency inventories and dynamic scans failed to connect the vulnerable Struts parser to the live `/struts2-rest-showcase` endpoint ($1.4B+ loss) [S1, S12].	**Disconnected SAST, DAST, and SCA reports:** SAST cries wolf on safe decoys; DAST can't point to the code line; SCA flags 85% unreachable CVEs while missing the 1 reachable RCE.	**Deterministic SAST+DAST Route-to-Sink Correlation (`FR-4` , `FR-5`) + `B-1` Auto-Verify + `B-2` Decoy Killer + 5-Step Reachability-Aware SCA (`Page 4 Bonus`).**

Geopolitical lens (global + India)

- **Sovereign Indian Cyber Resilience (*Atmanirbhar* AppSec & DPI Protection):** As India scales population-wide Digital Public Infrastructure (UPI, ONDC, Account Aggregator, ABDM) and sovereign AI under the **IndiaAI Mission**, relying on $150,000/year foreign ASPM/AppSec platforms is financially and strategically unviable for India's mid-market and regulated BFSI ecosystem.
- **Global Supply-Chain Warfare:** Between 2024 and 2026, state-sponsored and criminal groups shifted from zero-day exploits to **open-source supply-chain attacks** (malicious PyPI/npm packages, typosquatting, dependency confusion, and weaponizing newly disclosed GHSA commits within hours)—directly explaining why Anuj added the 5-step **Reachability-Aware SCA + Supply-Chain Hygiene** bonus to Page 4 of the Flagship PRD [S2].

What the organisers want to announce afterwards

Mitigata wants to announce that its Flagship Hackathon produced a breakthrough **Autonomous Code-to-Runtime Correlation & Reachability Engine**—accelerating `Gordon AI (GORDON-V3)` 's `/security/dast-sast` and `/gordon-ai/vapt` capabilities to achieve **sub-minute automated VAPT verification with 0% false alarms and function-level SCA reachability** across Indian enterprises.

Part 4 — Connecting the Dots

Dots nobody else will connect

1. **Dot 1 — Every Hackathon Problem Statement Is an Active R&D Epic Inside `GORDON-V3`:** Overlaying `Hackathon Problem Statements.pdf` onto `https://mitigata.com/gordon-ai` reveals a 1-to-1 product map: `PROBLEM 01` = Gordon AI SOC Kill-Chain Narratives; `PROBLEM 02` = Gordon Multi-Tenant Posture Console (`800+ clients`); `PROBLEM 03` = `/personal` Scam & Brand Protection; `PROBLEM 04` = Gordon Autonomous VAPT (`ReLIQ`); `PROBLEM 05` = Gordon's 200+ GRC/OEM Connectors; and **`PROBLEM 06 · FLAGSHIP`** = the core **`Assess`** engine where the #1 hero alert on `/gordon-ai` is literally `01 · SQL injection · /api/v2/users · Critical · CWE-89 · public endpoint · CVSS 9.8` [S1, S5, S6, S7].
2. **Dot 2 — The `38 Mean Findings` Fingerprint on `/gordon-ai/vapt` vs. `36 Planted Bugs` in Anuj's PRD:** On `https://mitigata.com/gordon-ai/vapt`, Mitigata advertises **`0 False positives`** and **`38 Mean findings`** [S7]. On Page 3 of `PRD_SAST_DAST_Correlation.docx.pdf`, the kickoff web-shop benchmark has **`36 planted vulnerabilities`** (`20 pairs + 10 code-only + 6 runtime-only`) and **`~8 safe decoys`** [S2]. The hackathon test app is directly modeled on Mitigata's internal VAPT benchmark!
3. **Dot 3 — Charlie Munger's Math Catch on `Wrong links: Under 5%` (`1 Error = Failure`):** Look at Page 4 of `PRD_SAST_DAST_Correlation.docx.pdf`: **`Wrong links`** is defined as "*Share of your Confirmed issues that join two different bugs*" with `Great: Under 5%` [S2]. Because there are only **20 true pairs** in the test app, if your tool reports 20 `Confirmed` issues and **even ONE (`1`)** of them joins two different bugs (e.g. joining a SQLi on `/api/orders?id=` with a SQLi on `/api/orders?sort=`), your wrong-link rate is `1 / 20 = 5.0%` (or `1 / 18 = 5.56%`), which **FAILS `Under 5%`**! Our correlator (`FR-5`) must enforce **strictly ZERO (`0`) wrong links (`0.0%`)** via deterministic `(Canonical_Route, HTTP_Method, Parameter_or_Sink_Flow, CWE)` matching and hard negative vetoes.
4. **Dot 4 — The `Runtime detection: Over 80%` + `6 Runtime only` Trap:** Page 3 states that **6 of the 36 planted bugs exist only at runtime (e.g. missing security headers)**, leaving 26 total runtime bugs (`20 pairs + 6 runtime-only`) [S2]. If a team's DAST scanner *only* probes SAST-extracted route parameters, it misses all 6 `Runtime only` bugs (capping runtime detection at `20/26 = 76.9%` if it misses even 1 active route, failing `Over 80%`!). Therefore, DAST **must** run in **Two Phases**: **Phase A (AST-Seeded Active Prober + `B-1` Auto-Verifier)** for the 20 pairs, and **Phase B (HTTP Crawler + Passive Header/Cookie/CORS/Config Auditor)** for the 6 `Runtime only` bugs (`FR-7`).
5. **Dot 5 — The `~8 Safe Decoys` Trap (`False alarms: Under 10%` Makes Bonus `B-2` Mandatory):** Page 3 warns that the test app contains **~8 safe decoys** ("*code that look dangerous but are safe, such as a properly parameterised database query. Reporting one is a false alarm*") [S2]. If a scanner reports 30 real bugs + 8 decoys, its false-alarm rate is `8 / 38 = 21.1%`, failing `Great: Under 10%`! Implementing Bonus `B-2` ("*mark findings as safe when the code is protected e.g. a parameterised query or no URL can reach it*") via an **AST Protection & Reachability Verifier** is mandatory to hit `0%` false alarms.
6. **Dot 6 — Why Bonus `B-1` (`Auto-verify`) Is the Secret Weapon for `Correct links: Over 80%`:** Standard black-box DAST crawlers miss unlinked POST endpoints, specific query parameter names, or required headers, leaving real pairs stranded as `Code only`. By implementing `B-1` ("*for a Code only*

finding, send a safe test request to try to prove it at runtime and upgrade it to Confirmed"), every `Code only` finding located on a reachable route (`FR-4`) triggers a surgical HTTP probe using the exact route, method, and parameter name extracted from the AST—pushing `Correct links` past 90%!

7. **Dot 7 — Page 4's Reachability-Aware SCA Bonus Gives the Exact Fix for GHSA's Missing Symbols:** Step 3 of the Page 4 SCA note explicitly states: "*3. Function map: Build it from OSV symbols and fix-commit diffs, since GHSA rarely lists functions.*" [S2]. By building a **GitHub Fix-Commit Unified Diff Parser** (`@@ ... @@ def <vulnerable_func>`) alongside OSV symbol lookup, tracing the `FR-4` entry-point-to-library call graph (`Step 4`), checking typosquats/dependency confusion/unpinned lockfiles (`Step 5`), and **never downgrading** `Unknown` , we solve the exact problem Anuj highlighted.
8. **Dot 8 — Answering Page 5's Final Question (`"Which language should this target first?"`):** Page 5's standalone question is a test of thoroughness [S2]. Every code example in the PRD (`orders.py line 88` , `get_orders()` , `Python's ast module`) proves the kickoff web shop is **Python** (Flask / FastAPI), while Bonus `B-7` rewards supporting a second language/framework (**JavaScript / Node.js Express** via `tree-sitter`).
9. **Dot 9 — Inter-Procedural Helper Calls in `FR-4` & `FR-5` :** While the simple example on Page 2 shows line 88 directly inside `get_orders()` (`lines 80 to 95`), several of the 20 planted pairs in Test App 1 or the Unseen App will place the SQL/Command/Path sink inside a helper function called by the route handler (e.g., `get_orders()` calls `db.query_orders(order_id)`). Tracing a 1-to-3 hop intra-project call graph from every route handler ensures `FR-5` correlates multi-function bugs effortlessly.
10. **Dot 10 — Connecting `FR-9` Risk Score to Mitigata's Cyber Insurance Underwriting Moat:** Combining technical exploitability (`Confirmed > Runtime only > Code only`) with Mitigata's FAIR `₹` financial exposure model (`/gordon-ai/cyber-risk-quantification`) and **SEBI CSCRF 3-month closure SLAs** turns `FR-9` into an executive decision engine [S2, S8, S9].

Hidden evaluation criteria

1. **Zero Hardcoding on the Unseen Second App (`Page 4 Target: Over 65%`):** At judging, the organisers run your tool against an unseen second app. Any hardcoded route strings (`/api/orders`), filenames (`orders.py`), or line numbers will score `0%` . Route templates (`/orders/<int:id>` , `/orders/{id}` , `/orders/:id`) must be normalized dynamically into canonical `{param}` slots.
2. **Strict Idempotency (`FR-6`):** Judges will run `POST /scans` or the CLI command **twice in a row** on the same target to verify that "*running a scan twice does not create copies*" and that "*3 scanner findings on the same bug = 1 issue*" [S2].
3. **Sub-5-Minute Cold-Start Execution (`Page 4 Target: Under 5 minutes`):** The entire pipeline—route mapping, SAST, decoy filtering, DAST probing, `B-1` auto-verification, correlation, SCA reachability, and report generation—must finish in under 5 minutes (our target: **25–40 seconds**).
4. **README Open-Source Disclosure (`Page 1 & Page 4 Rules`):** "*Open-source scanners and rule sets are allowed, but list them in the README. The correlation logic must be your own work.*" [S2].

Landmines — what will annoy this jury

1. **Disqualification Landmine — Outbound Network Requests During DAST (`Page 4 Rule`):** If your DAST scanner tests Open Redirect (`CWE-601`) with `redirect=https://example.com` and your HTTP client follows redirects (`follow_redirects=True`), it will send traffic outside the local test container,

violating "*Point the DAST scanner only at the organisers' app running on your own machine. Scanning any other site or system is not allowed and leads to disqualification.*" [S2]. **Fix:** Enforce a strict target host/port allowlist and `follow_redirects=False`.

2. **Joining Same-CWE Bugs on Different Endpoints or Parameters (`Wrong links >= 5%`):** Linking `DAST CWE-89 on /api/products?q=` with `SAST CWE-89 in orders.py:88` just because both are `CWE-89` destroys your `Wrong links` score.
3. **Reporting the 8 Safe Decoys (`False alarms >= 10%`):** Failing to check whether a SQL query uses parameterized bound parameters (`cursor.execute(sql, (id,))`) or whether HTML output is escaped.
4. **Downgrading `Unknown` in Reachability-Aware SCA:** Page 4 explicitly instructs: `Statuses: Reachable, Imported-only, Present-only, Unknown (never downgrade Unknown)` [S2]. Any tool that marks a CVE as safe when function symbol metadata is missing fails the SCA rubric.

The 80% trap — what most teams will build

- **What 80% of teams will build:** A bash script or FastAPI wrapper that runs `semgrep --config auto` and `zap-baseline.py`, dumps both JSON outputs into an LLM prompt (`"Match these SAST and DAST findings"`), and renders a basic React table.
- **Why it collapses at judging:** 1. **Fails `FR-4`:** It never reads the source code to build an explicit `URL -> function (start_line..end_line)` Route Map. 2. **Fails `Wrong links < 5%`:** The LLM guesses links between same-CWE findings across different routes/parameters, producing 2–4 wrong links (`10%–20%` error rate). 3. **Fails `False alarms < 10%`:** Stock Semgrep/Bandit flags the ~8 safe decoys because no `B-2` AST guard verifier filters them out. 4. **Fails `Correct links > 80%` & `Run time < 5 min`:** Blind ZAP crawling misses unlinked API endpoints/POST parameters and takes 8–12 minutes. 5. **Ignores Page 4's 5-Step Reachability-Aware SCA:** At best runs `pip-audit` without OSV/fix-commit diff function mapping, call-graph tracing, or typosquat detection.

Part 5 — The Battlefield

Competition forecast

Track	Estimated Share of Teams	Default Approach Most Competitors Will Take	Why the Field Will Struggle
`PROBLEM 06 · FLAGSHIP` — **SAST & DAST Correlation**	~25%	LLM wrapper over raw Semgrep + ZAP JSON outputs	Crushed by the 7 quantitative answer-key targets (`Wrong links < 5%` , `False alarms < 10%` , `Unseen app > 65%`) and 5-step SCA reachability
`PROBLEM 03` — **Community Scam Map**	~30%	CRUD web app + Google Maps / Leaflet + basic upvote counter	Overcrowded beginner tarpit; most fail `<0.5s` lookup at scale and anti-report-bombing trust math (`03 Beat fake reports`)
`PROBLEM 04` — **Autonomous Pentest Engine (CART)**	~15%	LangChain/CrewAI wrapper around `nmap` and `nuclei`	Multi-agent loops get stuck, scan hosts twice (violating `02 Coordinate agents`), or fail binary inspection
`PROBLEM 01` — **Kill Chain & Attack Path Analysis**	~12%	Loading CSV/JSON logs into Pandas/ Elasticsearch + LLM summary	Crashes with Out-Of-Memory (`OOM`) on the `30 GB` and `50 GB` log tiers (`02 Connect the steps without running out of memory`)
`PROBLEM 05` — **Config-Driven Integration Platform**	~10%	Simple YAML REST fetcher for 3 mock APIs	Lacks production rate-limit token buckets, exponential backoff jitter, secret isolation, and schema validation
`PROBLEM 02` — **Multi-Tenant Aggregation Layer**	~8%	Standard PostgreSQL `GROUP BY` + Redis cache	Either exceeds `<1s` on 10M rows or suffers cross-tenant cache key collisions (`INC-02`)

Prior work by domain

Domain / Track	Key Open-Source Repos & Prior Art	Commercial / Startup Benchmarks	Core Technical Gap We Exploit
`PROBLEM 06 · FLAGSHIP` **(SAST + DAST + SCA)**	`Semgrep` (~11k⭐), `Bandit` (~6.5k⭐), `OWASP ZAP` (~13k⭐), `Nuclei` (~21k⭐), `DefectDojo` (~3.9k⭐), `Joern` (~2.2k⭐), `osv-scanner` (~6.5k⭐) [S16–S21]	Mitigata `/security/dast-sast` & `/gordon-ai/vapt` , Apiiro, ArmorCode, Endor Labs, Socket.dev, ThreadFix	**No open-source tool builds an AST Route-to-Sink Call Graph (`FR-4`) to drive `B-1` DAST auto-verification, `B-2` decoy suppression, deterministic `FR-5` correlation, and 5-step OSV + fix-commit diff SCA reachability.**

Domain / Track	Key Open-Source Repos & Prior Art	Commercial / Startup Benchmarks	Core Technical Gap We Exploit
`PROBLEM 04` (CART Pentest Engine)	`PentestGPT`, `AutoAttacker`, `pwntools`, `angr`, `nuclei`, `nmap`	Mitigata `/gordon-ai/vapt` (`ReLIQ`), Horizon3.ai, XBOW, Terra Security	Deterministic blackboard state coordination across network, web, and binary agents with PTES attack-path proofs.
`PROBLEM 01` (Kill Chain 15–50 GB Logs)	`DuckDB`, `Polars`, `Sigma`, `Chainsaw`, `BloodHound`, `Graphistry`	Mitigata AI SOC (`/gordon-ai`), CrowdStrike Charlotte AI, Exabeam	Out-of-core streaming log ingestion + bounded-memory entity graph reconstruction + Markov next-hop prediction.
`PROBLEM 03` (Community Scam Map)	`H3` (Uber Hex Spatial Index), `Redis Bloom/Cuckoo Filter`	Chakshu (Sanchar Saathi), I4C `1930`, Truecaller	Sub-500ms Bloom/prefix lookup + H3 7-day spatio-temporal anomaly detection + Bayesian Sybil-resistant trust scoring.
`PROBLEM 02` (Multi-Tenant 10M Rollups)	`DuckDB`, `ClickHouse`, `TimescaleDB`, `PostgreSQL RLS`	Cube.dev, Tinybird, Citus	Incremental materialized delta rollups with strict `(tenant_id, hmac_epoch)` cache-key isolation and exact DB parity.
`PROBLEM 05` (Config-Driven Integrations)	`Airbyte CDK` (declarative YAML), `Meltano`, `Vector`, `Bentos`	Mitigata 200+ GRC Connectors, Merge.dev, Workato	Zero-code declarative YAML/JMESPath connector runtime with circuit breakers, DLQ, and live health telemetry.

Domain Winnability ranking

Computed using the 5-factor Phase 4 Winnability Index (`Competition Density [inv] 25% · Prior-Work Saturation [inv] 20% · Excitement/Demo 20% · Jury/Organiser Alignment 20% · Feasibility 15%`):

Rank	Problem Statement	Comp. Density (25%)	Prior-Work Gap (20%)	Demo / Wow (20%)	Jury & Organiser Alignment (20%)	Feasibility in Time Box (15%)	Winnability Score	3-Line Strategic Rationale
#1	`PROBLEM 06 · FLAGSHIP` **— SAST and DAST Correlation (+ Reachability SCA)**	9.2	9.6	9.7	**10.0**	9.2	**9.55 / 10**	**Explicitly badged** `FLAGSHIP · Solve it for bonus points` on Slide 7 and backed by Anuj's 5-page PRD [S1, S2]. Directly powers Mitigata's `/security/dast-sast` and `/gordon-ai/vapt` (`0 False positives` , `38 Mean findings`). High technical bar (`FR-4` , `B-1` , `B-2` , Page 4 SCA) wipes out shallow wrappers.
#2	`PROBLEM 04` **— Autonomous Pentest Engine (CART)**	8.5	8.2	9.4	9.2	7.3	**8.59 / 10**	Strong alignment with `/gordon-ai/vapt` offensive automation and dramatic PTES demo value. However, coordinating live network + web + binary exploitation in a hackathon environment carries higher live-demo fragility than PS-06's deterministic web shop benchmark.

Rank	Problem Statement	Comp. Density (25%)	Prior-Work Gap (20%)	Demo / Wow (20%)	Jury & Organiser Alignment (20%)	Feasibility in Time Box (15%)	**Winnability Score**	3-Line Strategic Rationale
#3	`PROBLEM 01` **— Kill Chain & Attack Path Analysis (15–50 GB Logs)**	8.4	8.0	8.5	8.8	7.2	**8.23 / 10**	Directly matches Gordon AI's AI SOC & Risk Narratives (`INC-01`). Streaming 50 GB without OOM using DuckDB/Polars + MITRE ATT&CK graph reconstruction is impressive, though lacks the Flagship bonus multiplier of PS-06.
#4	`PROBLEM 03` **— Community Scam Map for India**	4.5	7.0	9.0	8.2	9.5	**7.53 / 10**	High social resonance (`INC-03` Digital Arrest scams) and fast to prototype. Severely penalized by competition density: ~30% of teams will pick this track, turning judging into a crowded UI beauty contest.
#5	`PROBLEM 02` **— Multi-Tenant Aggregation Layer (10M Records)**	8.6	6.8	6.2	7.8	8.0	**7.50 / 10**	Essential backend infrastructure for Gordon AI's multi-tenant console (`INC-02`), and low competition density. However, sub-second SQL rollup benchmarks have lower visual stage excitement in a 3-minute pitch.

Rank	Problem Statement	Comp. Density (25%)	Prior-Work Gap (20%)	Demo / Wow (20%)	Jury & Organiser Alignment (20%)	Feasibility in Time Box (15%)	Winnability Score	3-Line Strategic Rationale
#6	`PROBLEM 05` **— Config-Driven Integration Platform**	8.8	6.5	6.0	7.5	7.4	**7.29 / 10**	Solves a real GRC connector pain (`INC-05`), and few teams will choose it. Ranked #6 because config-driven ETL plumbing is the hardest track to make visually unforgettable in 180 seconds.

- **Council's #1 Recommended Domain + Idea:** `PROBLEM 06 · FLAGSHIP` **+ Idea 1:** `GORDON CORRELATE (CorrelAid Engine)` **(Score: 9.60 / 10)** — The complete deterministic AST Route-to-Sink Call-Graph (`FR-4`) + 2-Phase Authenticated DAST (`FR-2`) + `B-1` Active Auto-Verifier + `B-2` Safe-Decoy Killer + `B-3` – `B-7` + 5-Step Reachability-Aware SCA (`Page 4 Bonus`) + Gordon AI FAIR `₹` Risk & SEBI CSCRF panel.
- **Council's Dark Horse Pick:** `PROBLEM 06 · FLAGSHIP` **+ Idea 4:** `GORDON REACH-SCA` **(Score: 9.29 / 10)** — Deep-tech execution of Page 4's 5-step Reachability-Aware SCA bonus (Lockfile Transitive SBOM → GHSA/CVSS/EPSS/KEV → OSV Symbols + GitHub Fix-Commit Diff Function Map → Entry-to-Library Call Graph → Supply-Chain Typosquat/Confusion Scanner), built as Layer 6 inside `GORDON CORRELATE` .

Part 6 — The Council

Who sat in the war room

To dominate `PROBLEM 06 · FLAGSHIP`, we assembled a **25-seat War-Room Council**: all **17 permanent core & extra seats** from `references/council-roster.md` plus **8 custom Domain & Mitigata Executive Titans** (pairing Mitigata's founders and PRD author lens with the world's foremost creators of SAST, DAST, CPG, and SCA tools, fully documented with 6-field dossiers in `war-room/roster-additions.md`):

Seat #	Person	Category & Credentials	Jurisdiction in the War Room	Mode on PROBLEM 06
01	**Peter Thiel**	Core · Co-founder PayPal, Palantir, Founders Fund	Contrarian secret (`Correlation is a Route-to-Sink Graph problem, not an LLM prompt problem`), 0-to-1 monopoly	**LEAD**
02	**Alex Karp**	Core · Co-founder & CEO, Palantir	30-second ontology pitch (*Code-to-Runtime-to-Rupee Truth*, preventing `INC-06` Equifax)	**LEAD**
03	**Patrick Jane** ◇	Core · Master Mind-Reader (*The Mentalist*)	Reverse-engineering PRD author `Anuj` , the 5 math traps on Pages 3–4, and the 80% competitor trap	**LEAD (Chair — Recon)**
04	**Warren Buffett**	Core · Chairman & CEO, Berkshire Hathaway	Cyber insurance float, underwriting loss-ratio moat, `₹` unit economics for Mitigata's 800+ clients	**LEAD**
05	**Harvey Specter** ◇	Core · Legal Closer (*Suits*)	Strict target-scope lock (`Page 4 Disqualification Rule` , `follow_redirects=False`), safe-harbour probing	**LEAD (Veto)**
06	**Mike Ross** ◇	Core · Human Conscience (*Suits*)	Developer/AppSec alert-fatigue empathy, composite practitioner stories, ethics of `Unknown` SCA safety	**LEAD (Veto)**
07	**Steve Jobs** †	Core · Co-founder, Apple	45-second live demo choreography (`36 planted bugs -> 20 Confirmed -> 0 Wrong links -> 0 False alarms`)	**LEAD**
08	**Elon Musk**	Core · Founder, SpaceX, Tesla, xAI	First-principles `< 35s` scan speed (deleting slow JVM/browser bloat) & Year-10 autonomous CI/CD moonshot	**LEAD**
09	**Andrej Karpathy**	Core · Founding Member OpenAI, ex-Director AI Tesla	Deterministic `ast` / `tree-sitter` + HTTP canary verification first; LLM strictly bounded to `B-4` fix text	**LEAD (Veto)**

Seat #	Person	Category & Credentials	Jurisdiction in the War Room	Mode on PROBLEM 06
10	**Dr. S. Jaishankar**	Core · External Affairs Minister, India	Sovereign *Atmanirbhar* cyber resilience for Indian critical infrastructure (CERT-In / NCIIPC)	**LEAD**
11	**Dr. R.S. Sharma**	Core · Ex-DG UIDAI, Ex-Chair TRAI, Ex-CEO NHA	**SEBI CSCRF** (3-month closure / 5-month revalidation), **DPDP Act 2023 & Rules 2025**, **CERT-In 6-Hour Directions**	**LEAD (Veto)**
12	**Charlie Munger †**	Extra · Vice Chairman, Berkshire Hathaway	Inversion pre-mortem (`Wrong links < 5%` means `1/20` error fails; `8 safe decoys` ; `never downgrade Unknown`)	**LEAD (Veto)**
13	**Seth Godin**	Extra · Author (*Purple Cow*, *This Is Marketing*)	Category positioning: *The Zero-Noise Code-to-Runtime Proof Engine*	**LEAD**
14	**Gary Vaynerchuk**	Extra · CEO, VaynerMedia	Developer distribution via open-source `B-6` GitHub Actions SARIF PR gate	**LEAD**
15	**Tim Cook**	Extra · CEO, Apple	Operational execution, 1-command Docker Compose reliability, offline cache resilience for judging	**LEAD**
16	**Jony Ive**	Extra · Founder, LoveFrom; ex-CDO Apple	3-screen `GORDON-V3` -aligned dark console (`Prioritised List` , `FR-8 Split Inspector` , `SCA Call-Graph View`)	**LEAD**
17	**Paul Graham**	Extra · Co-founder, Y Combinator	Schlep filter & Hackathon Judge Simulator (chairs 9-criterion weighted rubric scoring)	**LEAD (Chair — Scoring)**
18	**Mohit Anand**	Custom · Co-Founder & CEO, Mitigata	Buyer-judge lens: unifying `Assess` (SAST/DAST), `Comply` (SEBI/DPDP), and `Insure` (IRDAI underwriting)	**LEAD**
19	**Sarthak Dubey** *(with Mayank Morya & Anuj Lens)*	Custom · Co-Founder & COO / CTO, Mitigata	`/gordon-ai/vapt` KPIs (`0 False positives` , `38 Mean findings`), `/security/dast-sast` (`MAP -> SELECT -> TUNE -> VERIFY`)	**LEAD**
20	**Isaac Evans** *(with Drew Dennison Lens)*	Custom · Co-Founders, Semgrep (r2c)	Fast `tree-sitter` & Python `ast` rules, taint tracking, and **`B-2` safe-decoy AST guard verification**	**LEAD**
21	**Oege de Moor** *(with Fabian Yamaguchi Lens)*	Custom · Creators of CodeQL (Semmle) & Joern CPG	**`FR-4` AST Route Map**, inter-procedural helper call graphs (`FR-5`), and Unified Code Property Graphs	**LEAD**
22	**Dafydd Stuttard** *(with Simon Bennetts "Psiinon" Lens)*	Custom · Creators of Burp Suite & OWASP ZAP	**2-Phase DAST (`FR-2`)**, **`B-1` active auto-verification**, and **6 `Runtime only` bug capture (`FR-7`)**	**LEAD**

Seat #	Person	Category & Credentials	Jurisdiction in the War Room	Mode on PROBLEM 06
23	**Oliver Chang** *(with Feross Aboukhadijeh Lens)*	Custom · Creators of Google OSV.dev & Socket.dev	**Page 4 5-Step Reachability-Aware SCA** (`OSV symbols + fix-commit diffs`) & **Supply-Chain typosquat/confusion guard**	**LEAD**
24	**Subho Halder** *(with Dan Cornell Lens)*	Custom · Co-Founder Appknox & Creator of ThreadFix	Multi-scanner normalization (`FR-3`), idempotent deduplication (`FR-6`), scan diffing (`B-5`), SARIF gate (`B-6`)	**LEAD**
25	**Sandeep Kamble** *(with Shubham Shah "Shubs" Lens)*	Custom · Founder SecureLayer7 & ProjectDiscovery Nuclei	Offensive template verification, route parameter normalization, **Unseen App (`>65%`)**, and **Polyglot** `B-7`	**LEAD**

Sources

1. **[S1] Mitigata Official Hackathon Deck (`Hackathon Problem Statements.pdf`):** `d:\Mitigata\Hackathon Problem Statements.pdf` (Slides `01 / 07` to `07 / 07`, defining `PROBLEM 01` through `PROBLEM 06 · FLAGSHIP` and `REAL INCIDENT · INC-01` to `INC-06`).
2. **[S2] Mitigata Flagship Engineering PRD (`PRD_SAST_DAST_Correlation.docx.pdf`):** `d:\Mitigata\PRD_SAST_DAST_Correlation.docx.pdf` ("*PRD: SAST and DAST Correlation — Problem, Goals and Requirements*", authored by `Anuj`, dated `30 October 2026`, Pages 1–5).
3. **[S3] Mitigata Problem Statements Summary (`Problem Statements.docx`):** `d:\Mitigata\Problem Statements.docx` (6-problem-statement initial draft, dated `29 October 2026`).
4. **[S4] Mitigata Official Platform & Insurtech Homepage:** https://mitigata.com/ — Smart Cyber Insurance, IRDAI Direct Broker Registration No. `1115/2024` (License `1013`), CIN `U66220KA2024PTC186580`, $15M Series B led by Bessemer Venture Partners, Forbes Asia 100 to Watch 2026.
5. **[S5] Mitigata Application Security Testing (DAST + SAST):** https://mitigata.com/security/dast-sast — "*Find code flaws before attackers do*" (`MAP -> SELECT -> TUNE -> VERIFY` pipeline).
6. **[S6] Mitigata Gordon AI Platform (`CONSOLE · GORDON-V3`):** https://mitigata.com/gordon-ai — Unified `Monitor · Assess · Mitigate · Insure` architecture and live `CWE-89` SQLi telemetry.
7. **[S7] Mitigata Gordon AI Autonomous VAPT:** https://mitigata.com/gordon-ai/vapt — `0 False positives`, `38 Mean findings`, `48h Report delivery`, `11d Mean time to fix` ("*Find it. Prove it. Fix it. Re-scan it.*").
8. **[S8] Mitigata Cyber Risk Quantification (FAIR Model in ₹):** https://mitigata.com/gordon-ai/cyber-risk-quantification — Financial exposure modeling in ₹ Crores and underwriting bindability.
9. **[S9] Mitigata SEBI CSCRF Compliance Automation:** https://mitigata.com/compliance/sebi-cscrf — Automated VAPT closure tracking and compliance mapping for SEBI Regulated Entities.
10. **[S10] Mitigata API Security & Personal Cyber Protection:** https://mitigata.com/resources/apis and https://mitigata.com/personal.
11. **[S11] Mitigata Leadership & Company Dossier:** https://mitigata.com/company/about-us — Founders Mohit Anand (CEO), Sarthak Dubey (COO), Mayank Morya (CTO), Akshit Kaushik (CBO), Lt Cdr Dheeraj Yadav (CISO), and Dr. Lalit Gupta (Senior Partner & CDO).
12. **[S12] U.S. House Committee on Oversight and Government Reform — *The Equifax Data Breach* (`INC-06`, CVE-2017-5638):** https://oversight.house.gov/report/the-equifax-data-breach/ and https://nvd.nist.gov/vuln/detail/CVE-2017-5638.
13. **[S13] SEBI Cyber Security and Cyber Resilience Framework (CSCRF) Circular (`SEBI/HO/ITD-1/ITD_CSC_EXT/P/CIR/2024/113`):** https://www.sebi.gov.in/ — Mandatory SAST/DAST/API VAPT, 3-month vulnerability closure, and 5-month revalidation audit rules.
14. **[S14] CERT-In Cyber Security Directions (28 April 2022):** https://www.cert-in.org.in/ — 6-hour incident reporting mandate and 180-day log retention in India.
15. **[S15] Digital Personal Data Protection (DPDP) Act, 2023 & DPDP Rules, 2025 (MeitY, Govt. of India):** https://www.meity.gov.in/ — Section 8(5) reasonable security safeguards and ₹250 Crore statutory penalty schedule.

16. **[S16] Semgrep OSS & Tree-Sitter AST Analysis:** https://semgrep.dev/docs/ and https://github.com/semgrep/semgrep.
17. **[S17] PyCQA Bandit & OWASP Zed Attack Proxy (ZAP):** https://bandit.readthedocs.io/ and https://www.zaproxy.org/.
18. **[S18] ProjectDiscovery Nuclei Template Engine:** https://github.com/projectdiscovery/nuclei.
19. **[S19] OWASP DefectDojo & Joern Code Property Graph (CPG):** https://github.com/DefectDojo/django-DefectDojo and https://joern.io/.
20. **[S20] Open Source Vulnerability (OSV) Schema, Google `osv-scanner` & GitHub Advisory Database (GHSA):** https://ossf.github.io/osv-schema/, https://github.com/google/osv-scanner, and https://github.com/github/advisory-database.
21. **[S21] FIRST Exploit Prediction Scoring System (EPSS) & CISA Known Exploited Vulnerabilities (KEV) Catalog:** https://www.first.org/epss/ and https://www.cisa.gov/known-exploited-vulnerabilities-catalog.
22. **[S22] Hybrid Static-Dynamic Vulnerability Correlation Research (*SAST-Genius*, arXiv 2024 / IEEE):** https://arxiv.org/abs/2409.13068.
23. **[S23] OASIS SARIF v2.1.0 Standard Specification (`B-6`):** https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html.
24. **[S24] MITRE CWE Benchmark Definitions (`CWE-89` , `CWE-79` , `CWE-78` , `CWE-22` , `CWE-601` , `CWE-798`):** https://cwe.mitre.org/.